

Photon Interactive — Backend Round · 2-Day Crash Prep

You: Priyanka R · 5 yrs · Full-Stack (Angular + Java/Spring Boot) · Provility Software Solutions **Round:** Backend technical — scheduled **Saturday (11 PM slot)** **Prep time you have:** ~2 hrs today · ~4 hrs tomorrow night (on the bus, Chennai → Erode)

Honest framing for the whole round: You are strong full-stack with real Java + Spring Boot production work (Robogebra, EasyVisa). You are *not* claiming to be a deep backend specialist — you are a full-stack engineer who ships backend features end-to-end. That is a **credible, honest position**. When unsure, say what you know, tie it to real code, and offer "here's how I'd find out / how I'd approach it." Interviewers reward that.

The stack — know these cold (this is the Robogebra backend)

Thing	Answer
Java version	Java 17 (LTS)
Framework	Spring Boot 3.2.0 (Spring 6), Maven build
Database	MongoDB (NoSQL, document store) — via Spring Data MongoDB
Auth	AWS Cognito (3 user pools) + JWT + OAuth2 (Google/LinkedIn)
Scheduling	Quartz (2.3.2) for cron jobs (study reminders, subscription activation)
Architecture	Microservices hybrid — robogebra-portal (Java, :3000) ↔ robogebra-crm (Node, :3001) over REST
Your projects	Robogebra (Java+Mongo), EasyVisa (Grails/Groovy+Postgres+Neo4j), Subsea (.NET)

If they ask "what Java version / Spring Boot version do you use?" → **"Java 17 with Spring Boot 3.2."** Instant credibility.

Your 2-day plan (time-boxed to what you actually have)

TODAY — 2 hours (at a desk, so use the code)

- (30 min)** Read **03-java.md** + **04-spring.md** in this pack. These are the highest-yield. Say the OOP + DI answers out loud.
- (30 min)** Read **02-backend.md** (request lifecycle) + **05-database.md** (MongoDB). Memorize the *request flow* diagram: **filter** → **controller** → **service** → **repository** → **DB** → **DTO out**.
- (30 min)** Open the actual Robogebra code for **one** feature (solution-visibility) and trace it: controller → service → persistence. Being able to say "*in my project, this file does X*" wins the round.
- (30 min)** Read this file's **Top 30 Q&A** below once, out loud.

TOMORROW — 4 hours on the bus (phone only, no compiler)

- This is **review + memorize**, not new learning. On the bus you can't run code, so focus on the **Top 30 Q&A** and the **60-sec intro** below.
- **(1 hr)** 60-sec intro + project pitch until it's automatic.
- **(2 hr)** Top 30 Q&A — cover the answer, say it from memory, check.
- **(1 hr)** Weak spots only: whichever of Java / Spring / Mongo / microservices you fumbled.

SATURDAY (before the 11 PM slot)

- 30 min: re-read only the **60-sec intro**, the **stack table**, and the **"if you don't know" scripts**. Then rest your voice and be calm.

The 60-second intro (memorize this — it sets the tone)

"I'm Priyanka, a full-stack developer with about 5 years at Provility Software Solutions. I work across the stack — Angular and TypeScript on the front end, and Java with Spring Boot on the back end, with MongoDB. My current project is Robogebra, an AI-driven math learning platform. There I've built end-to-end features: REST APIs for a study-reminder and study-list system, a solution-visibility module that controls which classes can see answers, and integration with an AI explanation engine. Before that I worked on EasyVisa, a visa-management system using Grails/Groovy with Postgres and Neo4j. I'm comfortable owning a feature from API design and data model all the way to the UI. My strongest growth area right now is deepening backend and system-design depth, which is exactly why I'm excited about this role."

Why it works: honest, specific, ends on a growth note that reframes "less backend depth" as ambition.

Top 30 most-likely Q&A (condensed, spoken answers)

Java core

- 1. What Java version and what Java 17 features do you use?** "Java 17 LTS. I use switch expressions with arrow syntax (no fall-through, can return a value), `Optional` to avoid null checks, `var` for local inference, `.toList()` on streams, and text blocks. I understand records (immutable data carriers) and sealed classes too."
- 2. Explain the 4 OOP principles with an example.** "Encapsulation — private fields, expose behavior; in Robogebra my model hides `status` behind `isEnabled()`. Abstraction — I program to interfaces; my Mongo repository is just an interface, Spring generates the impl. Inheritance — my three service-auth filters share one abstract base filter. Polymorphism — a `NotificationService` interface where a factory picks WhatsApp vs Email at runtime and the caller just calls `sendOtp()`."
- 3. Difference between `ArrayList` and `LinkedList`?** "`ArrayList` is array-backed — $O(1)$ random access, cache-friendly — my default. `LinkedList` is node-based, only better for frequent inserts at the head/middle. I almost always use `ArrayList`."
- 4. How does a `HashMap` work internally?** "Array of buckets. The key's `hashCode()` picks the bucket; `equals()` resolves collisions within it. Since Java 8, a bucket with many collisions converts from a linked list to a balanced tree, so worst case is $O(\log n)$. That's why keys must have correct `equals` / `hashCode`."
- 5. `equals()` and `hashCode()` contract?** "If two objects are equal, they must return the same `hashCode`. Always override both together. It matters for `HashMap`/`HashSet` keys — break it and lookups silently fail."

6. What are Java Streams? Give a real use. "A declarative way to process collections — `map` transforms, `filter` keeps, `collect` gathers, `anyMatch` short-circuits. Intermediates like `map/filter` are lazy; nothing runs until a terminal like `collect`. In Robogeбра I stream class IDs, `map` each to an update, and `collect` — and use `anyMatch` to check if a solution is enabled for *any* of a student's classes."

7. String vs StringBuilder ? Why is String immutable? "String is immutable — safe to share, cacheable, good hash key. Every 'change' makes a new String. For heavy concatenation in a loop I use `StringBuilder`, which is mutable and avoids creating many temporary objects."

8. What is the difference between checked and unchecked exceptions? "Checked exceptions must be declared or caught (compile-time, e.g. `IOException`). Unchecked (`RuntimeException` subclasses) don't — they signal programming errors. In my APIs I throw an unchecked `APIException` with an HTTP status and let a global handler turn it into a clean JSON error."

9. == vs .equals() ? "`==` compares references (same object in memory); `.equals()` compares logical value. For objects always use `.equals()`. For `String`, `==` can accidentally pass due to the string pool — a classic bug."

Spring & Spring Boot

10. What is Spring Boot and why use it over plain Spring? "Spring Boot is Spring plus auto-configuration, starter dependencies, and an embedded server. It removes most XML/boilerplate — I add `spring-boot-starter-web`, and it wires an embedded Tomcat, JSON mapping, etc. I just run a `main()` method; no external server to deploy a WAR to."

11. What is Dependency Injection / IoC? "Inversion of Control means the framework creates and wires objects, not me. Dependency Injection is how — Spring injects a service's dependencies through its constructor. I declare `private final` fields and let Spring supply them. Benefit: loose coupling and easy testing — I can `new` the service with mocks in a unit test."

12. Constructor vs field injection — which and why? "Constructor injection. It makes dependencies `final` and immutable, guarantees they're set, and makes the class testable without Spring. Field injection with `@Autowired` hides dependencies and can't be final. I use Lombok's `@AllArgsConstructor` to generate the constructor."

13. Explain @RestController, @Service, @Repository, @Component. "All are Spring-managed beans (`@Component` is the base). `@RestController` = a web controller that returns JSON. `@Service` marks business logic. `@Repository` marks data access and translates DB exceptions. It's about layering: controller → service → repository."

14. What does @Transactional do? "Wraps a method in a database transaction — commit on success, rollback on a runtime exception. I put it on service methods that must be all-or-nothing. Note it works via a proxy, so self-invocation inside the same class bypasses it."

15. How does request validation work? "I annotate the DTO fields — `@NotNull`, `@NotEmpty`, `@Valid` for nested objects — and put `@Valid` on the controller's `@RequestBody`. If validation fails, Spring throws `MethodArgumentNotValidException`, which my global handler turns into a 400 with the field messages."

16. What is @ControllerAdvice / global exception handling? "A single class annotated `@ControllerAdvice` with `@ExceptionHandler` methods catches exceptions across all controllers and returns a consistent error body — `{status, message, timestamp, path}`. So controllers stay clean and every error looks the same to the frontend."

17. What is a Filter / how does Spring Security fit in? "A servlet Filter runs before the controller, on every request. In Robogeбра a `OncePerRequestFilter` extracts the JWT, validates it (our SSO token or AWS Cognito), loads the user, sets the Spring `SecurityContext`, and puts the user on a request attribute the controller reads. Authentication is centralized in one place, not repeated in each controller."

Database / MongoDB

18. SQL vs NoSQL — why MongoDB? "SQL is relational — fixed schema, tables, joins, strong transactions. MongoDB is a document store — flexible JSON-like documents, easy to scale horizontally, great when the shape varies or nests. Robogeбра's content (chapters, exercises, questions, user progress) is naturally document-shaped, so Mongo fits."

19. How do you query MongoDB in Spring? "Spring Data MongoDB. I extend `MongoRepository` and declare method names like `findByKlassIdAndTargetId` — Spring generates the query. For complex queries I use `MongoTemplate` with `Criteria`, or `@Query`."

20. What is an index and why does it matter? "An index is a data structure that lets Mongo find documents without scanning the whole collection. Without it, queries do a full collection scan — slow at scale. In Robogeбра I use a **unique compound index** on `(klass_id, target_id)` so there's exactly one visibility rule per class-per-target, and upserts key off it."

21. What is a `@DBRef` / how do you model relationships in Mongo? "`@DBRef` links one document to another (like a foreign key). But it can cause N+1 lookups, so I use it carefully — often I just store the referenced ID and fetch in a batch when needed."

WAS / web-server basics (Web Application Server)

22. What is a Web Application Server / servlet container? "It's the runtime that receives HTTP requests and hands them to your Java web app — it manages the servlet lifecycle, threads, and connection handling. Tomcat is the classic example. Spring Boot **embeds** Tomcat, so I don't deploy a WAR to an external server — the server is inside my JAR and I just run it."

23. WAR vs JAR / embedded vs external server? "Traditionally you build a WAR and deploy it into an external Tomcat/JBoss. With Spring Boot I build a single executable **JAR** with an embedded server — simpler to run, containerize, and scale. That's the modern default."

24. What is a servlet, briefly? "A servlet is a Java class that handles HTTP requests/responses inside the container. Spring MVC's `DispatcherServlet` is one central servlet that routes every request to the right controller method. I rarely write raw servlets — the framework does it — but Filters and the `DispatcherServlet` are servlet-layer concepts."

Microservices basics

25. What are microservices vs a monolith? "A monolith is one deployable app; microservices split it into small, independently deployable services that own their data and talk over the network (REST/messaging). Pros: independent scaling and deployment, tech freedom, fault isolation. Cons: network complexity, distributed data, harder debugging."

26. How do your services communicate? "In Robogeбра the Java `portal` and the Node `crm` service talk over **REST**. For internal service-to-service calls we use a shared **service token** (a special header) and a filter that authorizes it — so those calls skip user JWT auth but are still secured."

27. How do you handle a service being down / failures? "Timeouts and clear error mapping so one service's failure returns a proper 503, not a hang. Conceptually, retries with backoff, and a circuit breaker (e.g. Resilience4j) to stop hammering a dead service. In our code I map third-party/service errors to typed HTTP statuses instead of leaking 500s."

28. How do services share or keep data consistent? "Each service owns its own database — no shared DB. When they need each other's data they call an API or pass IDs. For cross-service consistency you use eventual consistency and events rather than one big transaction."

AWS basics

29. Which AWS services have you used? "Mainly **Cognito** for authentication — Robogebra has three Cognito user pools (regular user, email user, institute user). I've also worked with **S3** for file storage and **SES/SNS** for email/SMS notifications. The signup flow creates a Cognito user, sends an OTP, and issues a JWT the frontend uses on every request."

30. What is JWT and how does auth work end-to-end? "JWT is a signed token with the user's identity and claims. On login the backend (via Cognito or our SSO) issues it; the frontend stores it and sends it as a **Bearer** header on every request; a backend filter validates the signature/expiry, loads the user, and sets the security context. It's stateless — the server doesn't keep a session."

"If you don't know" — safe scripts (use these, don't freeze)

- **Don't know at all:** "I haven't used that directly, but my understanding is [X]. In my project the closest thing I've done is [real example]. I'd read the docs / spike it to confirm."
- **Half know:** "Let me reason it out — [think aloud]. I believe it's [X] because [why]."
- **Front-end pull:** you're allowed to shine here. "That's actually where I'm strongest — for example on the Angular side I..." Then bridge back.
- **Honesty on depth:** "I'm a full-stack engineer who ships backend features end-to-end rather than a backend specialist — but I own the API, the data model, and the logic for my features."

Quick reference — annotations (say what each does)

Annotation	One-liner
<code>@RestController</code>	Web controller returning JSON
<code>@RequestMapping</code> / <code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PatchMapping</code>	Route HTTP verb + path to a method
<code>@RequestBody</code> / <code>@RequestParam</code> / <code>@PathVariable</code>	Bind JSON body / query param / URL segment
<code>@Valid</code>	Trigger bean validation on a DTO
<code>@Service</code> / <code>@Repository</code> / <code>@Component</code>	Spring-managed beans by layer
<code>@Autowired</code> / constructor injection	Inject dependencies (prefer constructor)
<code>@Transactional</code>	Wrap method in a DB transaction
<code>@ControllerAdvice</code> + <code>@ExceptionHandler</code>	Global error handling
<code>@Document</code> / <code>@Id</code> / <code>@Indexed</code> (Mongo)	Map a class to a Mongo collection / id / index
Lombok <code>@Getter/@Setter/@Builder/@AllArgsConstructor</code>	Generate boilerplate

Quick reference — HTTP status codes

200 OK · 201 Created · 400 Bad Request (validation/business error) · 401 Unauthorized (not logged in) · 403 Forbidden (logged in, not allowed) · 404 Not Found · 409 Conflict (duplicate) · 500 Server Error · 503 Service Unavailable (downstream down)

Final reminders

- **Tie every answer to real code** ("in Robogebra, the solution-visibility service does X"). This is your edge.
- **Think out loud.** They score reasoning, not just the final word.
- **It's fine to say "I don't know, but here's how I'd find out."** Never bluff.
- **Breathe, slow down.** An 11 PM slot means you'll be tired — speak slower than feels natural.

You've got real 5-year full-stack experience shipping Java + Spring Boot features. Walk in owning that. Good luck! 🚀